Entities, Mapping and Repositories

8 min read Updated 28 Jul 2026 Topic 56 of 65

Bookmark Mark done

Why it matters

JPA is where a lot of Java applications quietly go wrong, and the mistakes are made at mapping time — months before anyone notices. Interviewers ask about entity design because the answers reveal whether you have maintained a persistence layer or only generated one: what `equals` should do on an entity, why bidirectional relationships need care, and why a record cannot be an entity.

The concept, simply

JPA is a specification; **Hibernate** is the implementation Spring Boot uses; **Spring Data JPA** sits on top and generates repository implementations from interfaces. Keeping those three apart is worth a mark on its own.

JAVA Copy

```java
@Entity
@Table(name = "orders")
public class Order {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, length = 32)
    private String reference;

    @Enumerated(EnumType.STRING)          // ✓ never ORDINAL — see below
    private OrderStatus status;
```

```
    @ManyToOne(fetch = FetchType.LAZY)      // ✓ LAZY by default on to-one
    @JoinColumn(name = "customer_id")
    private Customer customer;

    @OneToMany(mappedBy = "order", cascade = CascadeType.ALL, orphanRemova
    private List<OrderLine> lines = new ArrayList<>();

    protected Order() { }                   // required no-arg constructor
}
```

Why an entity cannot be a record: JPA requires a no-arg constructor, mutable fields for the provider to populate, and a non-final class so Hibernate can subclass it for lazy proxies. Records give none of those. Records are excellent for the DTOs you return *from* the repository layer — which is the right shape anyway.

How it works

Spring Data derives queries from method names, parsing the name into a query at startup. Get the name wrong and the application fails to start — which is much better than failing at run time.

JAVA

Copy

```
public interface OrderRepository extends JpaRepository<Order, Long> {

    // Derived — parsed from the name
    Optional<Order> findByReference(String reference);
    List<Order> findByStatusAndCreatedAtAfter(OrderStatus status, Instant
    long countByStatus(OrderStatus status);
    boolean existsByReference(String reference);

    // Explicit when the name would get silly
    @Query("SELECT o FROM Order o JOIN FETCH o.customer WHERE o.status = :
    List<Order> findByStatusWithCustomer(@Param("status") OrderStatus stat

    // Projections — fetch only the columns you need, mapped to a record
    interface OrderSummary { String getReference(); OrderStatus getStatus(
    List<OrderSummary> findSummaryByStatus(OrderStatus status);

    // ⚠ A modifying query bypasses the persistence context, so entities
```

```
    //   already loaded become stale. clearAutomatically fixes that.
    @Modifying(clearAutomatically = true)
    @Query("UPDATE Order o SET o.status = :status WHERE o.id = :id")
    int updateStatus(@Param("id") Long id, @Param("status") OrderStatus st
}
```

Bidirectional relationships need a helper. Setting only one side leaves the object graph inconsistent with what gets written, because the owning side — the one *without* `mappedBy` — is what JPA persists.

JAVA Copy

```
// ✓ Keep both sides in sync in one place.
public void addLine(OrderLine line) {
    lines.add(line);
    line.setOrder(this);        // the owning side - without this, order_i
}
public void removeLine(OrderLine line) {
    lines.remove(line);
    line.setOrder(null);
}
```

Code examples

The two mapping decisions that cause production incidents:

JAVA Copy

```
// 1. ✗ @Enumerated defaults to ORDINAL - it stores the enum's POSITION.
//    Insert a new constant in the middle and every existing row silently
//    means something different. There is no error, just wrong data.
@Enumerated                                  // ✗ ORDINAL by default
private OrderStatus status;

@Enumerated(EnumType.STRING)                 // ✓ always
private OrderStatus status;

// 2. ✗ EAGER fetching cannot be turned off per query, so it taxes every
//    query that touches the entity - including ones that never use it.
```

`equals` and `hashCode` on an entity — genuinely subtle, and Chapter 2 explains why:

JAVA Copy

```java
// ✗ Using a generated id: it is null before persist, so an entity added t
//   a HashSet before saving becomes unreachable once the id is assigned —
//   Chapter 2's mutable-key trap, in the wild.
// ✗ Using all fields: they change, so the hash changes.
// ✓ Use a business key that is assigned at construction and never changes
@Entity
public class Order {
    @Column(nullable = false, unique = true, updatable = false)
    private String reference = UUID.randomUUID().toString();

    @Override public boolean equals(Object o) {
        return o instanceof Order other && reference.equals(other.referenc
    }
    @Override public int hashCode() { return reference.hashCode(); }
}
// ⚠ Use instanceof rather than getClass(): Hibernate hands you a PROXY
//   subclass for a lazy association, so getClass() comparison always fail
```

Under the hood

Spring Data generates the implementation at runtime — a proxy backed by `SimpleJpaRepository`, with derived methods parsed from their names during context startup. That startup parsing is a feature: a typo in a property name breaks the build-out immediately with a clear message, rather than at the first call.

`save()` **is not an insert.** It calls `persist` for a new entity and `merge` for a detached one, deciding by whether the id is null (or via `Persistable`). And for an entity that is already *managed*, calling `save()` is redundant — dirty checking will write the change anyway. Understanding that is the difference between using JPA and fighting it.

lazy proxy without a query, which is ideal when you only need to set a foreign key. Touch any other property outside a transaction and you get `LazyInitializationException`.

Common mistakes

- `@Enumerated` **without** `STRING`**.** Ordinal storage corrupts data when constants are reordered.
- **Leaving** `@ManyToOne` **eager.** It is the default, and it taxes every query.
- `equals` **on a generated id.** Null before persist; breaks hash collections.
- `getClass()` **in an entity's** `equals`**.** Fails against Hibernate proxies — use `instanceof`.
- **Setting only one side of a bidirectional relationship.** The owning side is what persists.
- **Returning entities from controllers.** Leaks the schema and triggers lazy loading during serialisation.
- **Using a record as an entity.** No no-arg constructor, no mutability, final class.
- `@Modifying` **without** `clearAutomatically`**.** Loaded entities go stale.

Interview lens

> **Q: "JPA, Hibernate and Spring Data JPA — how do they relate?"**
> A: JPA is the specification defining the annotations and `EntityManager` API. Hibernate is the implementation, and Boot's default. Spring Data JPA is a layer above that generates repository implementations from interfaces, derives queries from method names, and adds paging and projections. You can use JPA without Spring Data and Hibernate without JPA.

> **Q: "How should** `equals` **and** `hashCode` **work on an entity?"**
> A: Based on a business key assigned at construction and never changed — a UUID reference, say — not on the generated id and not on all fields. The generated id is

`getClass()`, because Hibernate hands you a proxy subclass for lazy associations and a class comparison would always fail.

Q: "Why `EnumType.STRING`?"

A: Because the default, `ORDINAL`, stores the enum constant's position. Insert or reorder a constant and every existing row silently changes meaning — no exception, just corrupted data discovered much later. `STRING` costs a little space and is safe against reordering.

Q: "What does `save()` actually do?"

A: It calls `persist` for a new entity and `merge` for a detached one, choosing based on whether the id is null. For an entity that is already managed inside a transaction it is redundant — dirty checking issues the update at flush regardless. That surprises people who assume nothing is written without an explicit save.

Q: "Can a record be a JPA entity?"

A: No. JPA needs a no-arg constructor, mutable fields to populate, and a non-final class so Hibernate can create lazy proxies — records provide none of those. They are ideal as DTOs and projections returned from the repository layer, which is a better boundary anyway than passing entities upward.

Q: "`findById` or `getReferenceById`?"

A: `findById` queries immediately and returns an `Optional`. `getReferenceById` returns a lazy proxy with no query, which is exactly right when you only need the reference to set a foreign key on another entity. The catch is that touching any other property outside an active transaction throws `LazyInitializationException`.

- JPA = spec · Hibernate = implementation · Spring Data JPA = repository generation.
- **Always** `@Enumerated(EnumType.STRING)` — ordinal corrupts on reorder.
- `@ManyToOne` / `@OneToOne` **are EAGER by default** → set `LAZY` and fetch-join per query.
- `equals` / `hashCode` on a **business key**, with `instanceof` (proxies!) — never the generated id.
- Bidirectional: sync **both sides**; the owning side (no `mappedBy`) is what persists.
- `save()` = persist-or-merge; redundant on a **managed** entity (dirty checking).
- `getReferenceById` = lazy proxy, no query. `findById` = real query.
- **Records cannot be entities** — use them as DTOs/projections.
- Derived queries are parsed **at startup** — typos fail fast.

QUICK CHECK — no marks, just for you.

1. Why must you always write @Enumerated(EnumType.STRING)?

- ◯ Because ORDINAL cannot handle more than 10 constants
- ◯ Because ORDINAL is not supported by all databases
- ◯ Because the default, ORDINAL, stores the enum's POSITION — so inserting or reordering a constant silently changes the meaning of every existing row
- ◯ Because STRING is faster to index

Check answer

2. What should an entity's equals() be based on?

- ◯ The generated id

- Reference identity, inherited from Object
- A business key assigned at construction and never changed, compared with instanceof rather than getClass()

Check answer

3. What does repository.save() do for an entity that is already MANAGED inside a transaction?

- Nothing useful — it is redundant, because dirty checking issues the UPDATE at flush regardless
- It forces an immediate INSERT
- It detaches the entity
- It throws, because the entity already exists

Check answer

‹ Previous
Spring MVC: The Request Flow

Next ›
The Persistence Context and Entity States

LEARN MORE

ARTICLES & DOCS

Spring Data JPA reference
docs.spring.io

VIDEOS



"jpa entities and repositories" — video explainers

YouTube



jpa entities and repositories interview questions

YouTube

Hand-picked references for this lesson.